

ObjektinisProg

Sugeneruota Doxygen 1.17.0

1 ObjektinisProg	1
1.1 Naudojimosi instrukcija	1
1.2 Duomenų įvestis ir išvestis	1
1.2.1 Įvestis rankiniu būdu (1 parinktis)	1
1.2.2 Įvestis automatinio būdu (2 ir 3 parinktis)	1
1.2.3 Įvestis iš failo (4 parinktis)	2
1.2.4 Išvestis į ekraną (2 išvedimo būdas)	2
1.2.5 Išvestis į failą (1 ir 3 išvedimo būdas)	2
1.3 Perdengtų operatorių aprašas	2
1.3.1 operator<< — išvesties operatorius	2
1.3.2 operator>> — įvesties operatorius	2
1.4 Kompiliavimas	2
1.4.1 Testų kompiliavimas ir paleidimas	3
1.5 Klasės ir Struct Kompiliavimo flag'ų testas	3
1.5.1 100 000 studentų	3
1.5.2 1 000 000 studentų	3
1.6 Vientų testai	3
1.6.1 Testų paskirtis	4
1.7 Spartos tyrimas	4
1.7.1 1 strategija	4
1.7.2 2 strategija	5
1.7.3 3 strategija	5
1.8 Išvados	6
1.9 Dokumentacija	6
1.9.1 Dokumentacijos generavimas	6
1.10 Versijų istorija	6
2 Direktorijų hierarchija	7
2.1 Direktorijos	7
3 Hierarchijos Indeksas	9
3.1 Klasių hierarchija	9
4 Klasės Indeksas	11
4.1 Klasės	11
5 Failo Indeksas	13
5.1 Failai	13
6 Direktorijų dokumentacija	15
6.1 src Direktorijos aprašas	15
6.2 test Direktorijos aprašas	15
7 Klasės Dokumentacija	17

7.1 studentas Klasė	17
7.1.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	18
7.1.1.1 studentas() [1/4]	18
7.1.1.2 studentas() [2/4]	18
7.1.1.3 ~studentas()	18
7.1.1.4 studentas() [3/4]	18
7.1.1.5 studentas() [4/4]	18
7.1.2 Metodų Dokumentacija	18
7.1.2.1 addPaz()	18
7.1.2.2 clearPaz()	18
7.1.2.3 getEgz()	18
7.1.2.4 getGal()	19
7.1.2.5 getPavarde()	19
7.1.2.6 getPaz()	19
7.1.2.7 getRez()	19
7.1.2.8 getVardas()	19
7.1.2.9 med()	19
7.1.2.10 operator=() [1/2]	19
7.1.2.11 operator=() [2/2]	19
7.1.2.12 print()	19
7.1.2.13 setEgz()	19
7.1.2.14 setGal()	19
7.1.2.15 setPavarde()	19
7.1.2.16 setPaz()	20
7.1.2.17 setRez()	20
7.1.2.18 setVardas()	20
7.1.2.19 vid()	20
7.1.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija	20
7.1.3.1 operator<<	20
7.1.3.2 operator>>	20
7.2 Zmogus Klasė	20
7.2.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	21
7.2.1.1 Zmogus() [1/2]	21
7.2.1.2 Zmogus() [2/2]	21
7.2.1.3 ~Zmogus()	21
7.2.2 Metodų Dokumentacija	21
7.2.2.1 getPavarde()	21
7.2.2.2 getVardas()	21
7.2.2.3 print()	21
7.2.2.4 setPavarde()	21
7.2.2.5 setVardas()	21
7.2.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija	22

7.2.3.1 operator<<	22
7.2.4 Atributų Dokumentacija	22
7.2.4.1 pavarde	22
7.2.4.2 vardas	22
8 Failo Dokumentacija	23
8.1 README.md Failo Nuoroda	23
8.2 src/main.cpp Failo Nuoroda	23
8.2.1 Funkcijos Dokumentacija	23
8.2.1.1 main()	23
8.3 test/main.cpp Failo Nuoroda	23
8.3.1 Funkcijos Dokumentacija	24
8.3.1.1 sukurtiStudenta()	24
8.3.1.2 TEST() [1/18]	24
8.3.1.3 TEST() [2/18]	24
8.3.1.4 TEST() [3/18]	24
8.3.1.5 TEST() [4/18]	24
8.3.1.6 TEST() [5/18]	24
8.3.1.7 TEST() [6/18]	24
8.3.1.8 TEST() [7/18]	25
8.3.1.9 TEST() [8/18]	25
8.3.1.10 TEST() [9/18]	25
8.3.1.11 TEST() [10/18]	25
8.3.1.12 TEST() [11/18]	25
8.3.1.13 TEST() [12/18]	25
8.3.1.14 TEST() [13/18]	25
8.3.1.15 TEST() [14/18]	25
8.3.1.16 TEST() [15/18]	25
8.3.1.17 TEST() [16/18]	25
8.3.1.18 TEST() [17/18]	26
8.3.1.19 TEST() [18/18]	26
8.4 src/mylib.cpp Failo Nuoroda	26
8.4.1 Funkcijos Dokumentacija	26
8.4.1.1 generuotiFaila()	26
8.4.1.2 generuotiPazymius()	26
8.4.1.3 generuotiViska()	26
8.4.1.4 getFile()	27
8.4.1.5 getInt()	27
8.4.1.6 iverstiRanka()	27
8.4.1.7 pagalGal()	27
8.4.1.8 pagalPavard()	27
8.4.1.9 pagalVard()	27

8.4.1.10 printRez()	27
8.4.1.11 randomstr()	27
8.4.1.12 skaitytilsFailo()	27
8.4.1.13 skirstymas()	27
8.5 src/mylib.h Failo Nuoroda	28
8.5.1 Tipų apibrėžimų Dokumentacija	28
8.5.1.1 Studentai	28
8.5.2 Funkcijos Dokumentacija	28
8.5.2.1 generuotiFaila()	28
8.5.2.2 generuotiPazymius()	28
8.5.2.3 generuotiViska()	29
8.5.2.4 getFile()	29
8.5.2.5 getInt()	29
8.5.2.6 iverstiRanka()	29
8.5.2.7 pagalGal()	29
8.5.2.8 pagalPavard()	29
8.5.2.9 pagalVard()	29
8.5.2.10 printRez()	29
8.5.2.11 randomstr()	29
8.5.2.12 rusiuoti()	29
8.5.2.13 skaitytilsFailo()	30
8.5.2.14 skirstymas()	30
8.6 mylib.h	30
Rodyklė	33

skyrius 1

ObjektinisProg

1.1 Naudojimosi instrukcija

Funkcijos:

- Pasirinktinai išvestis į failą arba į terminalą.
- Pasirinktinai galutinio rezultato skaičiavimas, remiantis vidurkiu arba mediana.
- Rūšiavimas pasirinktinu būdu.
- Studentų skirstymas į „nevykelius“ (galutinis < 5) ir „nerdus“ (galutinis ≥ 5).

1 parinktis – rankinis duomenų įvedimas: studento vardo, pavardės, namų darbų rezultatų, egzamino rezultato.
2 parinktis – pusiau rankinis įvedimas: studento vardo, pavardės įvedimas, rezultatų generavimas.
3 parinktis – automatinis studentų vardų, pavardžių, rezultatų generavimas, jų apdorojimas ir išvedimas.
4 parinktis – skaitymas iš pasirinkto failo, rūšiavimas pasirinktinu būdu, duomenų apdorojimas ir išvedimas.
5 parinktis – studentų failų generavimas: studentų, namų darbų kiekio pasirinkimas ir išvedimas į studentų failą.
6 parinktis – programos nutraukimas.

Pasirinkus 1–4 parinktį, programa toliau klausia:

Rūšiavimo kriterijus	– pagal vardą, pavardę arba galutinį balą.
Skaiciavimo būdas	– galutinis balas skaičiuojamas vidurkiu arba mediana.
Išvedimo būdas	– į failą, į ekraną arba skirstymas į „nevykelių“ ir „nerdų“ failus.

1.2 Duomenų įvestis ir išvestis

1.2.1 Įvestis rankiniu būdu (1 parinktis)

Vartotojas įveda kiekvieno studento duomenis ranka. Programa prašo vardo, pavardės, namų darbų pažymių (baigiama įvedant -1) ir egzamino pažymio. Naudojama `getInt()` funkcija su validacija — neteisingi įvedimai atmami.

```
Įrašykite studento vardą (arba -1 baigti): Jonas
Įrašykite studento pavardę: Jonaitis
Įrašykite studento nd pažymį (arba -1 baigti): 8
Įrašykite studento nd pažymį (arba -1 baigti): 9
Įrašykite studento nd pažymį (arba -1 baigti): -1
Įrašykite studento egzamino pažymį: 10
```

1.2.2 Įvestis automatinio būdu (2 ir 3 parinktis)

2 parinktis — vartotojas įveda vardą ir pavardę, pažymiai generuojami atsitiktinai.

3 parinktis — viskas generuojama automatiškai: vardai, pavardės, pažymiai.

1.2.3 Įvestis iš failo (4 parinktys)

Programa nuskaityto studentų duomenis iš tekstinio failo. Failo formatas:

Vardas	Pavarde	ND1	ND2	... Egz.
Vardas1	Pavardel	7	8	... 9

Kiekviena eilutė nuskaityta per `operator>>`, kuris automatiškai išskiria vardą, pavardę, namų darbų pažymius ir egzaminą.

1.2.4 Išvestis į ekraną (2 išvedimo būdas)

Rezultatai spausdinami į terminalą per `operator<<`:

Vardas	Pavardė	Galutinis (Vid.)
-----	-----	-----
Jonas	Jonaitis	8.80

1.2.5 Išvestis į failą (1 ir 3 išvedimo būdas)

1 būdas — visi studentai išvedami į `isvedimas.txt`.

3 būdas — studentai skirstomi: nevykeliai į `nevykeliai.txt`, nerds į `nerds.txt`.

1.3 Perdengtų operatorių aprašas

`studentas` klasėje perdengiami du srautų operatoriai: `operator>>` (įvestis) ir `operator<<` (išvestis). Abu deklaruojami kaip `friend` funkcijos, nes turi tiesiogiai pasiekti `private` klasės laukus.

1.3.1 `operator<<` — išvesties operatorius

```
friend std::ostream& operator<<(std::ostream& out, const studentas& s);
```

Išveda studento vardą, pavardę ir galutinį balą suformatuoti. Grąžina `out`, kad būtų galimas grandininis išvedimas.

Naudojamas `printRez()` funkcijoje išvedant visą studentų sąrašą:

```
for (const auto& s : A)
    out << s << '\n';
```

Taip pat galima naudoti tiesiogiai:

```
studentas s("Jonas", "Jonaitis", {8, 9}, 10);
s.setGal(8.80);
std::cout << s;
// Jonas          Jonaitis          8.80
```

1.3.2 `operator>>` — įvesties operatorius

```
friend std::istream& operator>>(std::istream& in, studentas& s);
```

Nuskaityto vardą, pavardę, visus skaičius — paskutinis laikomas egzamino pažymiu, likusieji — namų darbų pažymiais. Grąžina `in` grandininiam skaitymui.

Naudojamas `skaitytiIsFailo()` funkcijoje:

```
std::istreamstream iss(line);
iss >> temp;
```

Taip pat galima naudoti tiesiogiai su `std::istreamstream`:

```
studentas s;
std::istreamstream iss("Jonas Jonaitis 8 9 10");
iss >> s;
// s.getVardas() == "Jonas", s.getEgz() == 10, paz == {8, 9}
```

1.4 Kompiliavimas

Reikalavimai: CMake 3.14, MinGW (Windows) arba GCC (Linux).

```
mkdir build
cd build
cmake .. -G "MinGW Makefiles" -DCONTAINER=vector # Windows
# cmake .. -DCONTAINER=vector # Linux
cmake --build .
```

Vietoje `vector` galima nurodyti `list` arba `deque`. Numatytoji reikšmė — `vector`.

1.4.1 Testų kompiliavimas ir paleidimas

```
cmake .. -G "MinGW Makefiles" -DCONTAINER=vector -DBUILD_TESTS=ON
cmake --build .
ctest --output-on-failure
# arba tiesiogiai:
./tests
```

1.5 Klasės ir Struct Kompiliavimo flag'ų testas

1.5.1 100 000 studentų

Optimizavimas	Versija	Vykdymo laikas (ms)	Failo dydis (ms)
O1	struct	873.15	1035
	class	657.62	1029
O2	struct	864.67	1036
	class	653.08	1026
O3	struct	854.61	1055
	class	641.19	1043

1.5.2 1 000 000 studentų

Optimizavimas	Versija	Vykdymo laikas (ms)	Failo dydis (ms)
O1	struct	9623.41	1035
	class	6617.74	1029
O2	struct	9543.10	1036
	class	6474.63	1026
O3	struct	9424.76	1055
	class	6518.39	1043

Rezultatai rodo, kad class visais atvejais veikė greičiau nei struct. Didinant optimizavimo lygį (O1 → O3), vykdymo laikas mažėjo, o failo dydis didėjo.

1.6 Vienetų testai

Vienetų testai ([test/main.cpp](#)) realizuoti naudojant **Google Test** karkasą. Testai tikrina pagrindinį [studentas](#) klasės funkcionalumą ir Rule of Five realizaciją. GTest atsisiunčiamas automatiškai per CMake `FetchContent` — rankinio diegimo nereikia.

Testuojamos šios dalys:

- Numatytasis konstruktorius.
- Kopijavimo konstruktorius.
- Kopijavimo priskyrimo operatorius.
- Perkėlimo konstruktorius.
- Perkėlimo priskyrimo operatorius.
- `vid()` ir `med()` funkcijos (įskaitant kraštinį atvejį — tuščias pažymių sąrašas).
- Galutinio balo skaičiavimas vidurkiu ir mediana.
- `skirstymas()` — teisingas skaidymas, visi vykeliai, visi nevykeliai, tuščias konteineris.

- `printRez()` — išvesties patikrinimas per `std::ostream`.

Sėkmingai praėjus testams išvedama:

```
[=====] Running 18 tests from 4 test suites.
[-----] 5 tests from StudentasKonstruktorius
[ RUN      ] StudentasKonstruktorius.Numatytasis
[ OK       ] StudentasKonstruktorius.Numatytasis
...
[=====] 18 tests from 4 test suites ran.
[ PASSED   ] 18 tests.
```

1.6.1 Testų paskirtis

Testai skirti užtikrinti, kad:

- Rule of Five metodai veikia korektiškai.
- Objektų kopijavimas nesukelia bendrų duomenų problemų.
- Perkėlus objektą jo būsena išlieka validi.
- Vidurkio ir medianos skaičiavimai grąžina teisingus rezultatus.
- `skirstymas()` teisingai atskiria nevykelius visais kraštiniais atvejais.

1.7 Spartos tyrimas

Tyrimui naudojami prieš tai sugeneruoti failai. Bandyti visi rūšiavimo ir galutinio skaičiavimo būdai, rezultatai išreikšti vidurkiu.

Specs: AMD Ryzen 5 3600 · HyperX DDR4 16GB · SSD 970 M.2 250GB

1.7.1 1 strategija

Bendro studentai konteinerio skaidymas į du naujus to paties tipo konteinerius: „nevykeliai“ ir „nerds“. Tokiu būdu tas pats studentas yra dvejuose konteineriuose: bendrame studentai ir viename iš suskaidytų.

Studentų kiekis	Konteineris	Nuskaitymas (ms)	Rūšiavimas (ms)	Skirstymas (ms)
1 000	vector	2.53	0.18	0.09
	deque	1.74	0.21	0.08
	list	2.07	0.09	0.11
10 000	vector	45.12	26.47	0.43
	deque	41.61	29.34	0.29
	list	49.38	13.02	0.37
100 000	vector	439.84	363.55	16.28
	deque	441.07	404.92	15.83
	list	451.29	171.84	24.51
1 000 000	vector	4397.41	4868.73	192.74
	deque	4418.56	5341.08	169.17
	list	4454.83	2207.16	220.39
10 000 000	vector	44284.↔ 6	62254.37	1880.62
	deque	44514.↔ 8	69897.44	1865.93

Studentų kiekis	Konteineris	Nuskaitymas (ms)	Rūšiavimas (ms)	Skirstymas (ms)
	list	44742.↔ 3	28843.91	2444.18

1.7.2 2 strategija

Bendro studentų konteinerio skaidymas panaudojant tik vieną naują konteinerį: „nevykeliai“. Tokiu būdu, jei studentas yra nevykelis, jį turime įkelti į naująjį „nevykelių“ konteinerį ir ištrinti iš bendro studentų konteinerio. Po šio žingsnio studentai konteineryje liks vien tik nerds.

Studentų kiekis	Konteineris	Nuskaitymas (ms)	Rūšiavimas (ms)	Skirstymas (ms)
1 000	vector	4.67	2.00	0.36
	deque	4.32	0.67	0.00
	list	5.67	0.99	0.33
10 000	vector	43.85	28.67	2.00
	deque	49.99	29.15	0.00
	list	45.86	14.33	0.99
100 000	vector	427.02	377.19	12.96
	deque	435.62	246.08	13.86
	list	450.60	186.96	21.33
1 000 000	vector	4294.58	5088.73	141.03
	deque	4357.09	5189.21	161.02
	list	4516.69	2378.67	259.93
10 000 000	vector	45101.00	64604.73	1559.45
	deque	42853.97	66047.37	1807.13
	list	45069.80	30505.13	2956.44

1.7.3 3 strategija

Tas pats kaip 2 strategija, tačiau skirstymui naudojamas `std::partition` — elementai perstumiami vietoje (in-place) vienu perėjimu, po to nevykeliai nukopijuojami į naują konteinerį ir ištrinami iš bendro. Skirtingai nuo 2 strategijos, kuri naudoja `copy_if` ir `remove_if` (du perėjimai), `partition` tai atlieka vienu perėjimu.

Studentų kiekis	Konteineris	Nuskaitymas (ms)	Rūšiavimas (ms)	Skirstymas (ms)
1 000	vector	2.57	0.53	0.00
	deque	8.10	0.00	0.00
	list	9.97	0.00	0.00
10 000	vector	41.54	21.68	0.00
	deque	43.23	20.37	0.00
	list	45.32	16.32	0.00
100 000	vector	444.37	280.55	3.97
	deque	428.07	305.00	3.35
	list	424.28	186.82	8.17
1 000 000	vector	4453.88	3671.65	26.63

Studentų kiekis	Konteineris	Nuskaitymas (ms)	Rūšiavimas (ms)	Skirstymas (ms)
	deque	4267.51	4031.19	65.79
	list	4239.82	2598.73	100.15
10 000 000	vector	44772.97	40601.10	234.93
	deque	42661.50	48562.40	695.92
	list	42550.20	33182.10	1073.19

1.8 Išvados

Nuskaitymo ir rūšiavimo rezultatai išlieka panašūs visose trijose strategijose, kaip ir tikėtasi — skiriasi tik skirstymo dalis. Aiškiausiai tai matyti lyginant 2 ir 3 strategiją: `partition` (3 strat.) skirstymas yra žymiai greitesnis už `copy_if + remove_if` (2 strat.), nes atlieka tik vieną perėjimą per konteinerį vietoj dviejų. Skirtumas ypač ryškus didesniuose failuose — pvz., 10M studentų su `vector`: 2 strategija ~1559 ms, 3 strategija ~235 ms.

1.9 Dokumentacija

Projekto dokumentacija sugeneruota naudojant **Doxygen** ir prieinama `docs/` kataloge.

- `docs/html/index.html` — naršyklėje peržiūrima HTML dokumentacija.
- `docs/latex/refman.pdf` — sukompiliuotas PDF.

1.9.1 Dokumentacijos generavimas

doxygen Doxyfile

1.10 Versijų istorija

Versija	Pakeitimai
v0.1	Pradinis variantas. Duomenų įvedimas ranka, C masyvo ir <code>std::vector</code> realizacijos, galutinio balo skaičiavimas vidurkiu arba mediana, išvedimas į ekraną.
v0.2	Pridėtas duomenų nuskaitymas iš failo, išvedimas į failą, rūšiavimas pagal vardą, pavardę arba galutinį balą. Projektas išskaidytas į kelis failus.
v0.3	Funkcijos perkeltos į antraštinį (.h) ir realizacijos (.cpp) failus. Pridėtas klaidų gaudymas (exception handling).
v0.4	Pridėtas failų generavimas, studentų skirstymas į „nevykelius“ ir „nerdus“, programos spartos tyrimas su 5 skirtingo dydžio failais.
v1.0	Trys atskiros realizacijos (vector, list, deque). Išbandytos 3 skirstymo strategijos. Pridėtas <code>CMakeLists.txt</code> .
v1.2	<code>struct studentas</code> pertvarkyta į <code>class studentas</code> su <code>private</code> laukais, getteriais ir setteriais. Realizuoti visi Rule of Five metodai (destruktorius, kopijavimo ir perkėlimo konstruktoriai, kopijavimo ir perkėlimo priskyrimo operatoriai). Perdengiami <code>operator<<</code> ir <code>operator>></code> įvesties/išvesties operatoriai. Pridėti vienetų testai (<code>test.cpp</code>).
v1.5	Pridėta abstrakti klasė <code>Zmogus</code> , iš kurios išvedama klasė <code>Studentas</code> .
v2.0	Bendras <code>src/</code> kodas visiems konteineriams — konteineris pasirenkamas per <code>-DCONTAINER=</code> CMake flagą. Vienetų testai perkelti į Google Test karkasą (18 testų). Pridėta Doxygen dokumentacija (HTML + PDF).

skyrius 2

Direktorių hierarchija

2.1 Direktorijos

src	15
main.cpp	23
mylib.cpp	26
mylib.h	28
test	15
main.cpp	23

skyrius 3

Hierarchijos Indeksas

3.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Zmogus	20
studentas	17

skyrius 4

Klasės Indeksas

4.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

studentas	17
Zmogus	20

skyrius 5

Failo Indeksas

5.1 Failai

Visų failų sąrašas su trumpais aprašymais:

src/ main.cpp	23
src/ mylib.cpp	26
src/ mylib.h	28
test/ main.cpp	23

skyrius 6

Direktorių dokumentacija

6.1 src Direktorijos aprašas

Failai

- failas [main.cpp](#)
- failas [mylib.cpp](#)
- failas [mylib.h](#)

6.2 test Direktorijos aprašas

Failai

- failas [main.cpp](#)

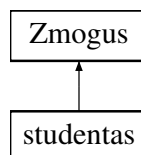
skyrius 7

Klasės Dokumentacija

7.1 studentas Klasė

```
#include <mylib.h>
```

Paveldimumo diagrama studentas:



Vieši Metodai

- `studentas ()`
- `studentas (const std::string &v, const std::string &p, const std::vector< int > &pazymiai, int e)`
- `~studentas ()` override
- `studentas (const studentas &other)`
- `studentas & operator= (const studentas &other)`
- `studentas (studentas &&other) noexcept`
- `studentas & operator= (studentas &&other) noexcept`
- `std::string getVardas ()` const override
- `std::string getPavarde ()` const override
- `void setVardas (const std::string &v)` override
- `void setPavarde (const std::string &p)` override
- `void print (std::ostream &out)` const override
- `std::vector< int > getPaz ()` const
- `int getEgz ()` const
- `double getRez ()` const
- `double getGal ()` const
- `void setPaz (const std::vector< int > &p)`
- `void addPaz (int p)`
- `void clearPaz ()`
- `void setEgz (int e)`
- `void setRez (double r)`
- `void setGal (double g)`
- `double vid ()` const
- `double med ()` const

Vieši Metodai inherited from **Zmogus**

- **Zmogus** ()
- **Zmogus** (const std::string &v, const std::string &p)
- virtual ~**Zmogus** ()

Draugai

- std::ostream & **operator<<** (std::ostream &out, const **studentas** &s)
- std::istream & **operator>>** (std::istream &in, **studentas** &s)

Additional Inherited Members

Apsaugoti Atributai inherited from **Zmogus**

- std::string **vardas**
- std::string **pavarde**

7.1.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.1.1.1 **studentas()** [1/4]

```
studentas::studentas () [inline]
```

7.1.1.2 **studentas()** [2/4]

```
studentas::studentas (
    const std::string & v,
    const std::string & p,
    const std::vector< int > & pazymiai,
    int e) [inline]
```

7.1.1.3 ~**studentas()**

```
studentas::~~studentas () [inline], [override]
```

7.1.1.4 **studentas()** [3/4]

```
studentas::studentas (
    const studentas & other) [inline]
```

7.1.1.5 **studentas()** [4/4]

```
studentas::studentas (
    studentas && other) [inline], [noexcept]
```

7.1.2 Metodų Dokumentacija

7.1.2.1 **addPaz()**

```
void studentas::addPaz (
    int p) [inline]
```

7.1.2.2 **clearPaz()**

```
void studentas::clearPaz () [inline]
```

7.1.2.3 **getEgz()**

```
int studentas::getEgz () const [inline]
```


7.1.2.4 getGal()

```
double studentas::getGal () const [inline]
```

7.1.2.5 getPavarde()

```
std::string studentas::getPavarde () const [inline], [override], [virtual]  
Realizuoja Zmogus.
```

7.1.2.6 getPaz()

```
std::vector< int > studentas::getPaz () const [inline]
```

7.1.2.7 getRez()

```
double studentas::getRez () const [inline]
```

7.1.2.8 getVardas()

```
std::string studentas::getVardas () const [inline], [override], [virtual]  
Realizuoja Zmogus.
```

7.1.2.9 med()

```
double studentas::med () const [inline]
```

7.1.2.10 operator=() [1/2]

```
studentas & studentas::operator= (  
    const studentas & other) [inline]
```

7.1.2.11 operator=() [2/2]

```
studentas & studentas::operator= (  
    studentas && other) [inline], [noexcept]
```

7.1.2.12 print()

```
void studentas::print (  
    std::ostream & out) const [inline], [override], [virtual]  
Realizuoja Zmogus.
```

7.1.2.13 setEgz()

```
void studentas::setEgz (  
    int e) [inline]
```

7.1.2.14 setGal()

```
void studentas::setGal (  
    double g) [inline]
```

7.1.2.15 setPavarde()

```
void studentas::setPavarde (  
    const std::string & p) [inline], [override], [virtual]  
Realizuoja Zmogus.
```

7.1.2.16 setPaz()

```
void studentas::setPaz (
    const std::vector< int > & p) [inline]
```

7.1.2.17 setRez()

```
void studentas::setRez (
    double r) [inline]
```

7.1.2.18 setVardas()

```
void studentas::setVardas (
    const std::string & v) [inline], [override], [virtual]
```

Realizuoja [Zmogus](#).

7.1.2.19 vid()

```
double studentas::vid () const [inline]
```

7.1.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija

7.1.3.1 operator<<

```
std::ostream & operator<< (
    std::ostream & out,
    const studentas & s) [friend]
```

7.1.3.2 operator>>

```
std::istream & operator>> (
    std::istream & in,
    studentas & s) [friend]
```

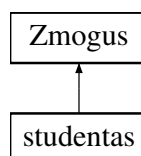
Dokumentacija šiai klasei sugeneruota iš šio failo:

- [src/mylib.h](#)

7.2 Zmogus Klasė

```
#include <mylib.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- [Zmogus](#) ()
- [Zmogus](#) (const std::string &v, const std::string &p)
- virtual [~Zmogus](#) ()
- virtual std::string [getVardas](#) () const =0
- virtual std::string [getPavarde](#) () const =0
- virtual void [setVardas](#) (const std::string &v)=0
- virtual void [setPavarde](#) (const std::string &p)=0
- virtual void [print](#) (std::ostream &out) const =0

Apsaugoti Atributai

- std::string [vardas](#)
- std::string [pavarde](#)

Draugai

- std::ostream & [operator<<](#) (std::ostream &out, const [Zmogus](#) &z)

7.2.1 Konstruktoriaus ir Destruktoriaus Dokumentacija**7.2.1.1 Zmogus() [1/2]**

```
Zmogus::Zmogus () [inline]
```

7.2.1.2 Zmogus() [2/2]

```
Zmogus::Zmogus (  
    const std::string & v,  
    const std::string & p) [inline]
```

7.2.1.3 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

7.2.2 Metodų Dokumentacija**7.2.2.1 getPavarde()**

```
virtual std::string Zmogus::getPavarde () const [pure virtual]  
Realizuota studentas.
```

7.2.2.2 getVardas()

```
virtual std::string Zmogus::getVardas () const [pure virtual]  
Realizuota studentas.
```

7.2.2.3 print()

```
virtual void Zmogus::print (  
    std::ostream & out) const [pure virtual]  
Realizuota studentas.
```

7.2.2.4 setPavarde()

```
virtual void Zmogus::setPavarde (  
    const std::string & p) [pure virtual]  
Realizuota studentas.
```

7.2.2.5 setVardas()

```
virtual void Zmogus::setVardas (  
    const std::string & v) [pure virtual]  
Realizuota studentas.
```

7.2.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija

7.2.3.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & out,  
    const Zmogus & z) [friend]
```

7.2.4 Atributų Dokumentacija

7.2.4.1 pavarde

```
std::string Zmogus::pavarde [protected]
```

7.2.4.2 vardas

```
std::string Zmogus::vardas [protected]
```

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [src/mylib.h](#)

skyrius 8

Failo Dokumentacija

8.1 README.md Failo Nuoroda

8.2 src/main.cpp Failo Nuoroda

```
#include <string>
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <cstdlib>
#include <fstream>
#include <sstream>
#include <chrono>
#include "mylib.h"
#include <windows.h>
```

Funkcijos

- int `main` ()

8.2.1 Funkcijos Dokumentacija

8.2.1.1 main()

```
int main ()
```

8.3 test/main.cpp Failo Nuoroda

```
#include <gtest/gtest.h>
#include "mylib.h"
#include <sstream>
```

Funkcijos

- `studentas sukurtiStudenta` (const std::string &v, const std::string &p, std::vector< int > pazymiai, int egz)
- `TEST` (StudentasKonstruktorius, Numatytasis)
- `TEST` (StudentasKonstruktorius, Kopijavimo)
- `TEST` (StudentasKonstruktorius, KopijavimoPreskyrimasOperatorius)
- `TEST` (StudentasKonstruktorius, Perkelimo)
- `TEST` (StudentasKonstruktorius, PerkelimoPreskyrimasOperatorius)
- `TEST` (Skaiciavimai, VidurklisVienas)

- **TEST** (Skaiciavimai, VidurklisKeli)
- **TEST** (Skaiciavimai, VidurklisOhnePazymiu)
- **TEST** (Skaiciavimai, MedianaNelyginisSkacius)
- **TEST** (Skaiciavimai, MedianaLyginisSkacius)
- **TEST** (Skaiciavimai, MedianaOhnePazymiu)
- **TEST** (Skaiciavimai, GalutinisVidurkiu)
- **TEST** (Skaiciavimai, GalutinisMediana)
- **TEST** (Skirstymas, NevykeliaiAtskiriami)
- **TEST** (Skirstymas, VisiVykeliai)
- **TEST** (Skirstymas, VisiNevykeliai)
- **TEST** (Skirstymas, TustiKonteineriaiNesugriuna)
- **TEST** (Isvedimas, PrintRezIsvedasTeisingai)

8.3.1 Funkcijos Dokumentacija

8.3.1.1 sukurtiStudenta()

```
studentas sukurtiStudenta (
    const std::string & v,
    const std::string & p,
    std::vector< int > pazymiai,
    int egz)
```

8.3.1.2 TEST() [1/18]

```
TEST (
    Isvedimas ,
    PrintRezIsvedasTeisingai )
```

8.3.1.3 TEST() [2/18]

```
TEST (
    Skaiciavimai ,
    GalutinisMediana )
```

8.3.1.4 TEST() [3/18]

```
TEST (
    Skaiciavimai ,
    GalutinisVidurkiu )
```

8.3.1.5 TEST() [4/18]

```
TEST (
    Skaiciavimai ,
    MedianaLyginisSkacius )
```

8.3.1.6 TEST() [5/18]

```
TEST (
    Skaiciavimai ,
    MedianaNelyginisSkacius )
```

8.3.1.7 TEST() [6/18]

```
TEST (
    Skaiciavimai ,
    MedianaOhnePazymiu )
```

8.3.1.8 TEST() [7/18]

```
TEST (
    Skaiciavimai ,
    VidurklisKeli )
```

8.3.1.9 TEST() [8/18]

```
TEST (
    Skaiciavimai ,
    VidurklisOhnePazymiu )
```

8.3.1.10 TEST() [9/18]

```
TEST (
    Skaiciavimai ,
    VidurklisVienas )
```

8.3.1.11 TEST() [10/18]

```
TEST (
    Skirstymas ,
    NevykeliaiAtskiriami )
```

8.3.1.12 TEST() [11/18]

```
TEST (
    Skirstymas ,
    TustiKonteineriaiNesugriuna )
```

8.3.1.13 TEST() [12/18]

```
TEST (
    Skirstymas ,
    VisiNevykeliai )
```

8.3.1.14 TEST() [13/18]

```
TEST (
    Skirstymas ,
    VisiVykeliai )
```

8.3.1.15 TEST() [14/18]

```
TEST (
    StudentasKonstruktorius ,
    Kopijavimo )
```

8.3.1.16 TEST() [15/18]

```
TEST (
    StudentasKonstruktorius ,
    KopijavimoPreskyrimasOperatorius )
```

8.3.1.17 TEST() [16/18]

```
TEST (
    StudentasKonstruktorius ,
    Numatytasis )
```

8.3.1.18 TEST() [17/18]

```
TEST (
    StudentasKonstruktorius ,
    Perkelimo )
```

8.3.1.19 TEST() [18/18]

```
TEST (
    StudentasKonstruktorius ,
    PerkelimoPreskyrimasOperatorius )
```

8.4 src/mylib.cpp Failo Nuoroda

```
#include "mylib.h"
#include <iostream>
#include <string>
#include <algorithm>
#include <fstream>
#include <stdexcept>
#include <iomanip>
#include <chrono>
#include <sstream>
```

Funkcijos

- std::string [randomstr](#) ()
- bool [pagalVard](#) (const [studentas](#) &a, const [studentas](#) &b)
- bool [pagalPavard](#) (const [studentas](#) &a, const [studentas](#) &b)
- bool [pagalGal](#) (const [studentas](#) &a, const [studentas](#) &b)
- int [getInt](#) (int min, int max)
- std::string [getFile](#) ()
- void [printRez](#) (std::ostream &out, [Studentai](#)< [studentas](#) > &A, int skaiciavimas)
- void [ivestiRanka](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [generuotiPazymius](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [generuotiViska](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [skaitytilsFailo](#) ([Studentai](#)< [studentas](#) > &A, int &m, std::string &failas)
- void [generuotiFaila](#) ()
- void [skirstymas](#) ([Studentai](#)< [studentas](#) > &studentai, [Studentai](#)< [studentas](#) > &nevykeliai)

8.4.1 Funkcijos Dokumentacija**8.4.1.1 generuotiFaila()**

```
void generuotiFaila ()
```

8.4.1.2 generuotiPazymius()

```
void generuotiPazymius (
    Studentai< studentas > & A,
    int & m)
```

8.4.1.3 generuotiViska()

```
void generuotiViska (
    Studentai< studentas > & A,
    int & m)
```


8.4.1.4 getFile()

```
std::string getFile ()
```

8.4.1.5 getInt()

```
int getInt (
    int min,
    int max)
```

8.4.1.6 ivestiRanka()

```
void ivestiRanka (
    Studentai< studentas > & A,
    int & m)
```

8.4.1.7 pagalGal()

```
bool pagalGal (
    const studentas & a,
    const studentas & b)
```

8.4.1.8 pagalPavard()

```
bool pagalPavard (
    const studentas & a,
    const studentas & b)
```

8.4.1.9 pagalVard()

```
bool pagalVard (
    const studentas & a,
    const studentas & b)
```

8.4.1.10 printRez()

```
void printRez (
    std::ostream & out,
    Studentai< studentas > & A,
    int skaiciavimas)
```

8.4.1.11 randomstr()

```
std::string randomstr ()
```

8.4.1.12 skaitytiIsFailo()

```
void skaitytiIsFailo (
    Studentai< studentas > & A,
    int & m,
    std::string & failas)
```

8.4.1.13 skirstymas()

```
void skirstymas (
    Studentai< studentas > & studentai,
    Studentai< studentas > & nevykeliai)
```

8.5 src/mylib.h Failo Nuoroda

```
#include <string>
#include <vector>
#include <list>
#include <deque>
#include <fstream>
#include <algorithm>
#include <iomanip>
```

Klasės

- class [Zmogus](#)
- class [studentas](#)

Tipų apibrėžimai

- template<typename T>
using [Studentai](#) = std::vector<T>

Funkcijos

- template<typename Container, typename Comp>
void [rusiuoti](#) (Container &c, Comp comp)
- std::string [randomstr](#) ()
- bool [pagalVard](#) (const [studentas](#) &a, const [studentas](#) &b)
- bool [pagalPavard](#) (const [studentas](#) &a, const [studentas](#) &b)
- bool [pagalGal](#) (const [studentas](#) &a, const [studentas](#) &b)
- int [getInt](#) (int min, int max)
- std::string [getFile](#) ()
- void [printRez](#) (std::ostream &out, [Studentai](#)< [studentas](#) > &A, int skaiciavimas)
- void [investiRanka](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [generuotiPazymius](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [generuotiViska](#) ([Studentai](#)< [studentas](#) > &A, int &m)
- void [skaitytilsFailo](#) ([Studentai](#)< [studentas](#) > &A, int &m, std::string &failas)
- void [generuotiFaila](#) ()
- void [skirstymas](#) ([Studentai](#)< [studentas](#) > &studentai, [Studentai](#)< [studentas](#) > &nevykeliai)

8.5.1 Tipų apibrėžimų Dokumentacija

8.5.1.1 Studentai

```
template<typename T>
using Studentai = std::vector<T>
```

8.5.2 Funkcijos Dokumentacija

8.5.2.1 generuotiFaila()

```
void generuotiFaila ()
```

8.5.2.2 generuotiPazymius()

```
void generuotiPazymius (
    Studentai< studentas > & A,
    int & m)
```

8.5.2.3 generuotiViska()

```
void generuotiViska (
    Studentai< studentas > & A,
    int & m)
```

8.5.2.4 getFile()

```
std::string getFile ()
```

8.5.2.5 getInt()

```
int getInt (
    int min,
    int max)
```

8.5.2.6 ivestiRanka()

```
void ivestiRanka (
    Studentai< studentas > & A,
    int & m)
```

8.5.2.7 pagalGal()

```
bool pagalGal (
    const studentas & a,
    const studentas & b)
```

8.5.2.8 pagalPavard()

```
bool pagalPavard (
    const studentas & a,
    const studentas & b)
```

8.5.2.9 pagalVard()

```
bool pagalVard (
    const studentas & a,
    const studentas & b)
```

8.5.2.10 printRez()

```
void printRez (
    std::ostream & out,
    Studentai< studentas > & A,
    int skaiciavimas)
```

8.5.2.11 randomstr()

```
std::string randomstr ()
```

8.5.2.12 rusiuoti()

```
template<typename Container, typename Comp>
void rusiuoti (
    Container & c,
    Comp comp)
```

8.5.2.13 skaitytilsFailo()

```
void skaitytilsFailo (
    Studentai< studentas > & A,
    int & m,
    std::string & failas)
```

8.5.2.14 skirstymas()

```
void skirstymas (
    Studentai< studentas > & studentai,
    Studentai< studentas > & nevykeliai)
```

8.6 mylib.h

Eiti į šio failo dokumentaciją.

```
00001 #pragma once
00002 #include <string>
00003 #include <vector>
00004 #include <list>
00005 #include <deque>
00006 #include <fstream>
00007 #include <algorithm>
00008 #include <iomanip>
00009
00010 #if defined(USE_LIST)
00011     template<typename T> using Studentai = std::list<T>;
00012 #elif defined(USE_DEQUE)
00013     template<typename T> using Studentai = std::deque<T>;
00014 #else
00015     template<typename T> using Studentai = std::vector<T>;
00016 #endif
00017
00018 class Zmogus {
00019 protected:
00020     std::string vardas;
00021     std::string pavarde;
00022
00023 public:
00024     Zmogus() : vardas(""), pavarde("") {}
00025     Zmogus(const std::string& v, const std::string& p) : vardas(v), pavarde(p) {}
00026
00027     virtual ~Zmogus() {}
00028
00029     virtual std::string getVardas() const = 0;
00030     virtual std::string getPavarde() const = 0;
00031     virtual void setVardas(const std::string& v) = 0;
00032     virtual void setPavarde(const std::string& p) = 0;
00033
00034     virtual void print(std::ostream& out) const = 0;
00035
00036     friend std::ostream& operator<<(std::ostream& out, const Zmogus& z) {
00037         z.print(out);
00038         return out;
00039     }
00040 };
00041
00042 class studentas : public Zmogus {
00043 private:
00044     std::vector<int> paz;
00045     int egz;
00046     double rez;
00047     double gal;
00048
00049 public:
00050     studentas()
00051         : Zmogus(), egz(0), rez(0.0), gal(0.0) {}
00052
00053     studentas(const std::string& v, const std::string& p,
00054               const std::vector<int>& pazymiai, int e)
00055         : Zmogus(v, p), paz(pazymiai), egz(e), rez(0.0), gal(0.0) {}
00056
00057     ~studentas() override {}
00058
00059     studentas(const studentas& other)
00060         : Zmogus(other.vardas, other.pavarde),
00061         paz(other.paz), egz(other.egz), rez(other.rez), gal(other.gal) {}
00062
00063     studentas& operator=(const studentas& other) {
```

```

00064         if (this == &other) return *this;
00065         vardas = other.vardas;
00066         pavarde = other.pavarde;
00067         paz = other.paz;
00068         egz = other.egz;
00069         rez = other.rez;
00070         gal = other.gal;
00071         return *this;
00072     }
00073
00074     studentas(studentas&& other) noexcept
00075         : Zmogus(std::move(other.vardas), std::move(other.pavarde)),
00076           paz(std::move(other.paz)), egz(other.egz), rez(other.rez), gal(other.gal) {
00077         other.vardas = ""; other.pavarde = "";
00078         other.egz = 0; other.rez = 0.0; other.gal = 0.0;
00079     }
00080
00081     studentas& operator=(studentas&& other) noexcept {
00082         if (this == &other) return *this;
00083         vardas = std::move(other.vardas);
00084         pavarde = std::move(other.pavarde);
00085         paz = std::move(other.paz);
00086         egz = other.egz; rez = other.rez; gal = other.gal;
00087         other.vardas = ""; other.pavarde = "";
00088         other.egz = 0; other.rez = 0.0; other.gal = 0.0;
00089         return *this;
00090     }
00091
00092     std::string getVardas() const override { return vardas; }
00093     std::string getPavarde() const override { return pavarde; }
00094     void setVardas(const std::string& v) override { vardas = v; }
00095     void setPavarde(const std::string& p) override { pavarde = p; }
00096
00097     void print(std::ostream& out) const override {
00098         out << std::left << std::setw(15) << vardas
00099             << '\t' << std::setw(15) << pavarde
00100             << '\t' << std::fixed << std::setprecision(2) << gal;
00101     }
00102
00103     std::vector<int> getPaz() const { return paz; }
00104     int getEgz() const { return egz; }
00105     double getRez() const { return rez; }
00106     double getGal() const { return gal; }
00107
00108     void setPaz(const std::vector<int>& p) { paz = p; }
00109     void addPaz(int p) { paz.push_back(p); }
00110     void clearPaz() { paz.clear(); }
00111     void setEgz(int e) { egz = e; }
00112     void setRez(double r) { rez = r; }
00113     void setGal(double g) { gal = g; }
00114
00115     friend std::ostream& operator<<(std::ostream& out, const studentas& s) {
00116         s.print(out);
00117         return out;
00118     }
00119
00120     friend std::istream& operator>>(std::istream& in, studentas& s) {
00121         s.paz.clear();
00122         in >> s.vardas >> s.pavarde;
00123         int x;
00124         while (in >> x) s.paz.push_back(x);
00125         s.egz = s.paz.back();
00126         s.paz.pop_back();
00127         return in;
00128     }
00129
00130     double vid() const {
00131         if (paz.empty()) return 0.0;
00132         double suma = 0;
00133         for (int p : paz) suma += p;
00134         return suma / paz.size();
00135     }
00136
00137     double med() const {
00138         if (paz.empty()) return 0.0;
00139         std::vector<int> sorted = paz;
00140         std::sort(sorted.begin(), sorted.end());
00141         int n = sorted.size();
00142         if (n % 2 == 0) return (sorted[n/2-1] + sorted[n/2]) / 2.0;
00143         else return sorted[n/2];
00144     }
00145 };
00146
00147 template<typename Container, typename Comp>
00148 void rusiuoti(Container& c, Comp comp) {
00149     #if defined(USE_LIST)
00150         c.sort(comp);

```

```
00151 #else
00152     std::sort(c.begin(), c.end(), comp);
00153 #endif
00154 }
00155
00156 std::string randomstr();
00157 bool pagalVard(const studentas& a, const studentas& b);
00158 bool pagalPavard(const studentas& a, const studentas& b);
00159 bool pagalGal(const studentas& a, const studentas& b);
00160 int getInt(int min, int max);
00161 std::string getFile();
00162 void printRez(std::ostream& out, Studentai<studentas>& A, int skaiciavimas);
00163 void ivestiRanka(Studentai<studentas>& A, int& m);
00164 void generuotiPazymius(Studentai<studentas>& A, int& m);
00165 void generuotiViska(Studentai<studentas>& A, int& m);
00166 void skaitytiIsFailo(Studentai<studentas>& A, int& m, std::string& failas);
00167 void generuotiFaila();
00168 void skirstymas(Studentai<studentas>& studentai, Studentai<studentas>& nevykeliai);
```

Rodyklė

- ~Zmogus
 - Zmogus, [21](#)
- ~studentas
 - studentas, [18](#)
- addPaz
 - studentas, [18](#)
- clearPaz
 - studentas, [18](#)
- generuotiFaila
 - mylib.cpp, [26](#)
 - mylib.h, [28](#)
- generuotiPazymius
 - mylib.cpp, [26](#)
 - mylib.h, [28](#)
- generuotiViska
 - mylib.cpp, [26](#)
 - mylib.h, [28](#)
- getEgz
 - studentas, [18](#)
- getFile
 - mylib.cpp, [26](#)
 - mylib.h, [29](#)
- getGal
 - studentas, [18](#)
- getInt
 - mylib.cpp, [27](#)
 - mylib.h, [29](#)
- getPavarde
 - studentas, [19](#)
 - Zmogus, [21](#)
- getPaz
 - studentas, [19](#)
- getRez
 - studentas, [19](#)
- getVardas
 - studentas, [19](#)
 - Zmogus, [21](#)
- investiRanka
 - mylib.cpp, [27](#)
 - mylib.h, [29](#)
- main
 - main.cpp, [23](#)
- main.cpp
 - main, [23](#)
 - sukurtiStudenta, [24](#)

- TEST, [24–26](#)
- med
 - studentas, [19](#)
- mylib.cpp
 - generuotiFaila, [26](#)
 - generuotiPazymius, [26](#)
 - generuotiViska, [26](#)
 - getFile, [26](#)
 - getInt, [27](#)
 - investiRanka, [27](#)
 - pagalGal, [27](#)
 - pagalPavard, [27](#)
 - pagalVard, [27](#)
 - printRez, [27](#)
 - randomstr, [27](#)
 - skaitytilsFailo, [27](#)
 - skirstymas, [27](#)
- mylib.h
 - generuotiFaila, [28](#)
 - generuotiPazymius, [28](#)
 - generuotiViska, [28](#)
 - getFile, [29](#)
 - getInt, [29](#)
 - investiRanka, [29](#)
 - pagalGal, [29](#)
 - pagalPavard, [29](#)
 - pagalVard, [29](#)
 - printRez, [29](#)
 - randomstr, [29](#)
 - rusiuoti, [29](#)
 - skaitytilsFailo, [29](#)
 - skirstymas, [30](#)
 - Studentai, [28](#)
- ObjektinisProg, [1](#)
- operator<<
 - studentas, [20](#)
 - Zmogus, [22](#)
- operator>>
 - studentas, [20](#)
- operator=
 - studentas, [19](#)
- pagalGal
 - mylib.cpp, [27](#)
 - mylib.h, [29](#)
- pagalPavard
 - mylib.cpp, [27](#)
 - mylib.h, [29](#)
- pagalVard

- mylib.cpp, 27
- mylib.h, 29
- pavarde
 - Zmogus, 22
- print
 - studentas, 19
 - Zmogus, 21
- printRez
 - mylib.cpp, 27
 - mylib.h, 29
- randomstr
 - mylib.cpp, 27
 - mylib.h, 29
- README.md, 23
- rusiuoti
 - mylib.h, 29
- setEgz
 - studentas, 19
- setGal
 - studentas, 19
- setPavarde
 - studentas, 19
 - Zmogus, 21
- setPaz
 - studentas, 19
- setRez
 - studentas, 20
- setVardas
 - studentas, 20
 - Zmogus, 21
- skaitytilsFailo
 - mylib.cpp, 27
 - mylib.h, 29
- skirstymas
 - mylib.cpp, 27
 - mylib.h, 30
- src Directorijos aprašas, 15
- src/main.cpp, 23
- src/mylib.cpp, 26
- src/mylib.h, 28, 30
- Studentai
 - mylib.h, 28
- studentas, 17
 - ~studentas, 18
 - addPaz, 18
 - clearPaz, 18
 - getEgz, 18
 - getGal, 18
 - getPavarde, 19
 - getPaz, 19
 - getRez, 19
 - getVardas, 19
 - med, 19
 - operator<<, 20
 - operator>>, 20
 - operator=, 19
 - print, 19
 - setEgz, 19
 - setGal, 19
 - setPavarde, 19
 - setPaz, 19
 - setRez, 20
 - setVardas, 20
 - studentas, 18
 - vid, 20
- sukurtiStudenta
 - main.cpp, 24
- TEST
 - main.cpp, 24–26
 - test Directorijos aprašas, 15
 - test/main.cpp, 23
- vardas
 - Zmogus, 22
- vid
 - studentas, 20
- Zmogus, 20
 - ~Zmogus, 21
 - getPavarde, 21
 - getVardas, 21
 - operator<<, 22
 - pavarde, 22
 - print, 21
 - setPavarde, 21
 - setVardas, 21
 - vardas, 22
 - Zmogus, 21